

Estimating NBA Shot Make Probability with a Stacked Ensemble and Player Skill Adjusted for Shot Difficulty

OKC Thunder Analyst Intern Project

Augustin de Menibus

September 29, 2026

1 Introduction

Which shots should a team take, and which shots should it allow? An offense wants the shots that produce the most points per attempt; a defense wants to push opponents toward the shots that produce the fewest. Both sides of that question rest on one quantity: the probability that a given shot goes in. Multiplying that probability by the shot’s point value gives its **expected points**, the common currency for comparing a corner three with a contested floater, judging shot selection, evaluating defenders and designing game plans.

This paper builds a model that estimates the make probability of any shot from player tracking data. The training data contain 425,719 regular season shots with known outcomes, each described by its location, its type, how closely it was defended, the game situation, and who took and defended it. The model then predicts a make probability for each of the 213,977 shots in a separate test file. Predictions are judged by **log loss**, which rewards honest probabilities rather than yes/no calls: a model that says 90% on a shot that misses is punished far more than one that says 55%. That shaped every decision in the project. The aim was never to “call” makes and misses, but to put the right number on every shot.

The main contributions are:

- **A leak-proof evaluation design.** An 80/20 split with five-fold cross-validation inside the 80%, where every cleaning step and every player statistic is learned inside the folds, so every reported score comes from shots the model had not seen.
- **Player skill adjusted for shot difficulty.** A “makes above expected” measure for shooters and defenders that separates skill from shot selection. It was the single most useful addition to the model.
- **A stacked ensemble of four different learners**, whose weights show which model the final predictions rely on.
- **A translation into basketball decisions**, turning the probabilities into expected points for offense and defense.

Section 2 describes the data, Section 3 the methods, Section 4 the results, and Section 5 how the model can be used. Sections 6 to 8 cover limitations, future work and conclusions.

2 Data

2.1 The dataset

The training file holds 425,719 non-fouled regular season shots from several recent seasons, with an overall make rate of 45.8% in the development set. No external data were used. The inputs fall into five groups.

Table 1: Input groups used by the model.

Group	Inputs
Location	distance, court coordinates (x, y), three-point attempt
Shot	shot type, dribbles before the shot, shot clock, shooter speed
Defense	closest defender distance at release and 1, 0.75, 0.5 and 0.25 s before; number of contesters and their distances; contested flag
Context	game state (half court, transition, second chance), month, season
Identity	shooter and closest defender (used only through the player skill columns)

2.2 Data cleaning

The cleaning ran in two stages. Steps that look at one shot at a time, without learning anything from the other shots, were done once on the whole file before the split, because they cannot leak information from one shot to another. Steps that learn something from the data, such as a median or the list of categories, were learned inside each cross-validation fold from that fold’s training shots only, and then applied unchanged to its validation shots (Section 3.1).

Stage 1: shot by shot, before the split.

1. **Checking the file.** I loaded `training.csv.gz`, checked the type and range of every column, and counted the missing values in each one (see the chart on missing values below).
2. **Categories.** Shot type and game state are labels with no natural order, so I stored them as unordered categories rather than numbers. Season and month were also treated as categories, and the true/false columns (contested and three-point attempt) became 0/1.
3. **Defender approach.** The closest defender’s approach is stored as one text field holding four distances in braces. I removed the braces, split the text at the commas and converted each value to a number, which gave four columns: the defender’s distance 1, 0.75, 0.5 and 0.25 seconds before release. The code stops with an error if a shot has anything other than four values, or a value that is not a number, and blank values stay missing. The original text column was then dropped.
4. **Inputs and outcome.** I kept only shots with a known outcome and the 22 inputs in the table above. The outcome, the shot id and the player ids are not inputs, and a check stops the code if any of them slips in. The player ids are used only to build the player skill columns (Section 3.3).

Stage 2: learned inside each fold. A `recipes` pipeline (Kuhn & Johnson, 2019) was learned on each fold’s training shots and applied to its validation shots, in the order below. For the holdout it was relearned on all development shots, and for the test file on all training shots.

Table 2: Cleaning steps learned inside each fold, in order.

Step	What it does	Why
New and missing categories	A category not seen in the training shots gets a “new” level, and a missing category gets an “unknown” level	The model can score every shot, even one from a season it never saw
Base categories	Dunk becomes the base category for shot type, and half court for game state	Explained below
Missing defender distances	Filled with the shot’s own distance	Explained below
Other gaps	Filled with the median of the training shots	Only a few columns still have gaps, such as the shot clock when it was turned off
Indicator columns	Each category becomes 0/1 columns, one fewer than it has levels	The base category is the shot where all of them are 0
Columns with no variation	Removed	Unused “new” and “unknown” levels carry no information
Exact duplicates	Removes any column that is an exact combination of others	The logistic regression cannot be fit with them
Standardizing	Rescales continuous columns to mean 0 and standard deviation 1	Puts inputs on one scale for the logistic regression and the neural network; trees are unaffected
Final check	Stops if any value is missing or infinite, or if the training and validation columns differ	Catches mistakes before any model is fit

Dunk as the base category. A category with k levels needs only $k - 1$ indicator columns. With a column for every shot type plus the model’s intercept, the shot type columns would always add up to the intercept, and the logistic regression could not be fit. One level therefore has to be the base: the shot where every shot type column is 0. I chose the dunk, the easiest shot in the data (made 88.5% of the time), so in the logistic regression every other shot type’s coefficient reads as how much harder that shot is than a dunk. Half court is the base for game state for the same reason: it is the usual situation, so transition and second-chance shots are measured against it. No information is lost, since every model still knows exactly which shots are dunks. The one side effect is that the dunk has no column of its own, so it cannot appear by name in the importance chart (Section 4.6), it shows up through distance and defender distance since we replace it with distance. Its effect shows up in the other shot type columns and in shot distance, since dunks happen at the rim.

Table 3: Make rate by shot type on the development shots.

Shot type	Shots	Share of shots	Make rate
Dunk (base category)	20,507	6.0%	88.5%
Layup	92,485	27.2%	53.5%

Shot type	Shots	Share of shots	Make rate
Tip	7,579	2.2%	48.0%
Post	11,840	3.5%	45.2%
Floater	22,175	6.5%	41.7%
Jumper	185,100	54.3%	37.9%
Heave	889	0.3%	9.7%

Dunks are the closest thing to a sure make because of the nature of the shot. You are as close as possible to the rim and defenders have a much harder time blocking it. Most missed dunks are due to extraordinary effort from defenders.

Missing defender distances. The chart below shows which inputs have gaps. Contester distances are missing whenever a shot has fewer contesters, and the closest defender’s distances are sometimes missing from tracking. When a defender distance was missing, I filled it with **the distance of the shot itself**. A missing defender usually means nobody was close enough to be tracked, so the shot was open; the defender most likely to be nearby is the one protecting the basket, roughly that far away. Any gaps left in other columns were filled with the median of the training shots.

Missing values by column

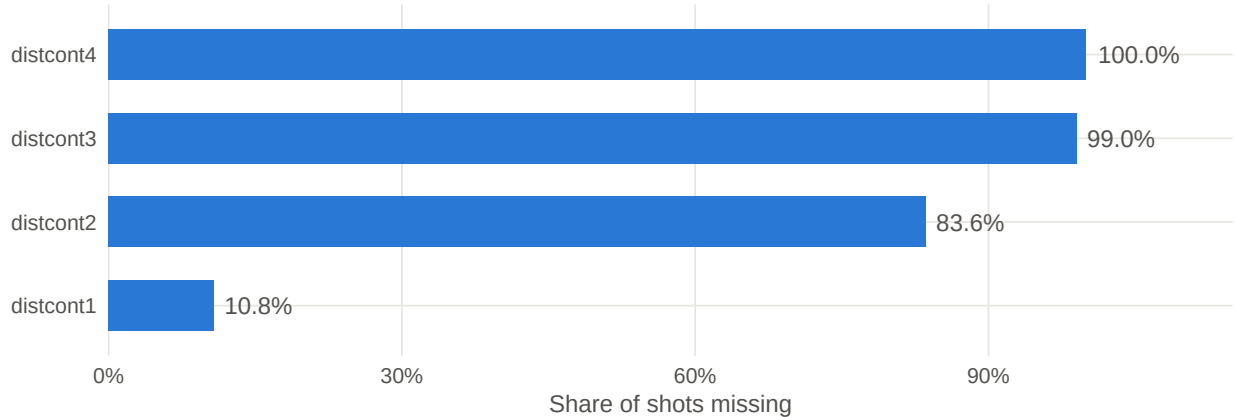


Figure 1: Share of shots with a missing value, for every column with gaps.

This rule has one side effect for the contester distances. When a shot has no third or fourth contester, the filled value is simply the shot distance, so those columns become copies of it: the fourth contester’s distance equals the shot distance on 100.0% of shots, and the third contester’s on 99.0%. The number of contesters tells the models which distances are real, so this probably costs little accuracy, but the copies share credit with shot distance in the importance chart (Section 4.6). In hindsight, a large value meaning “no contester” would have been a cleaner fill for these columns.

2.3 Exploratory analysis

Before modelling, I looked at how the make rate changes with the main shot features. The overall average hides very different shots.

Make rate by shot distance

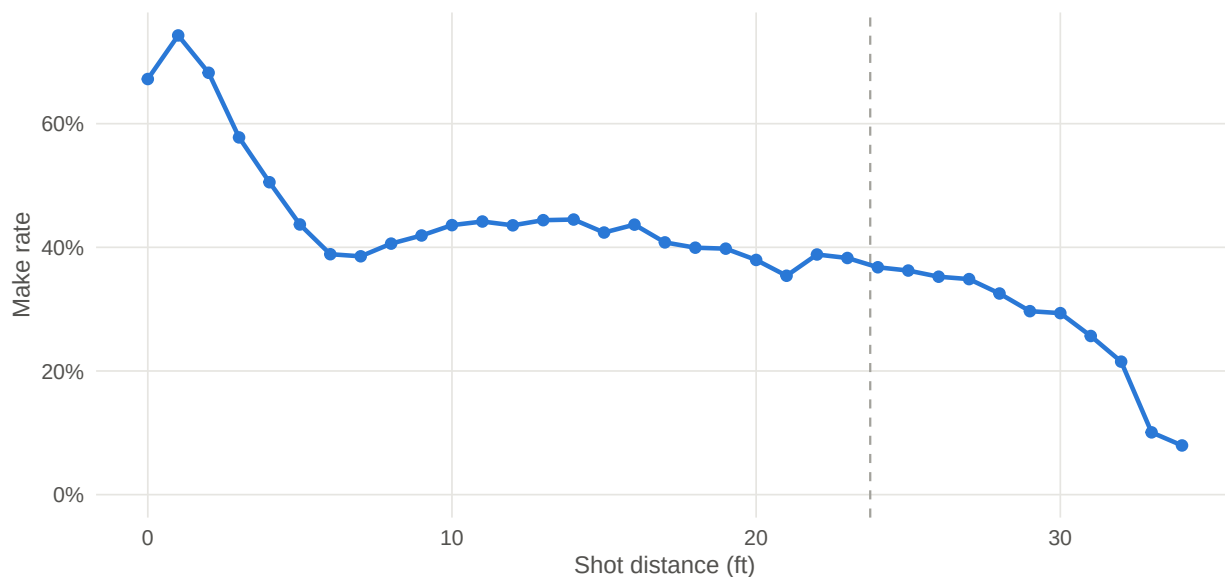


Figure 2: Make rate by shot distance. The dashed line marks the three-point line.

Distance is the strongest single driver. Shots at the rim go in far more often than anything else, and the make rate drops quickly over the first few feet. My exploration also showed lower make rates as the number of testers grows.

Make rate by time left on the shot clock

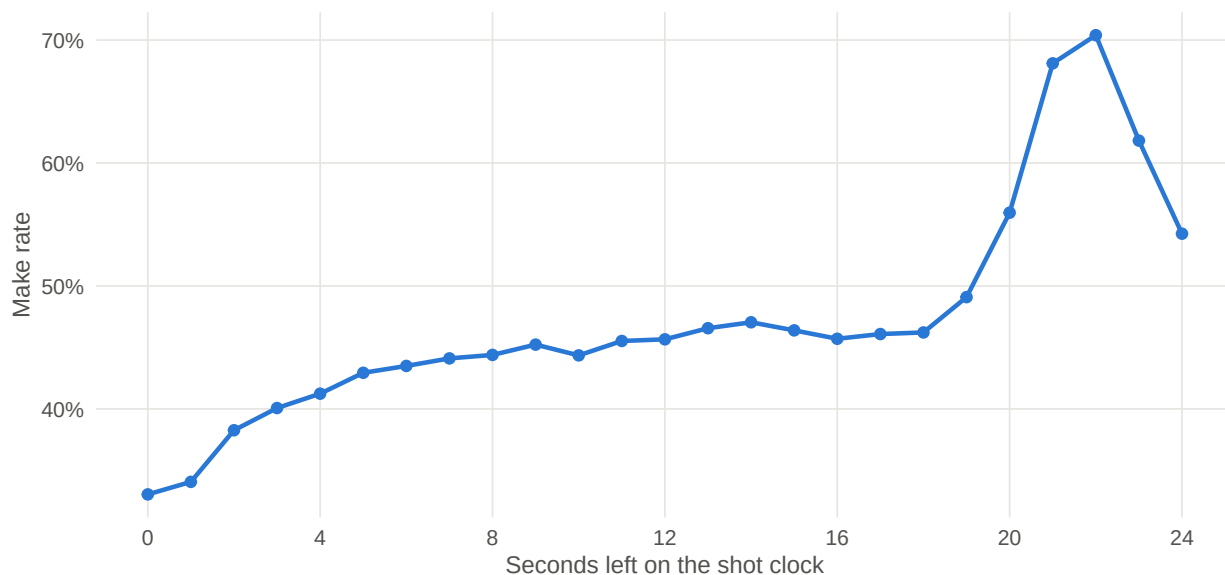


Figure 3: Make rate by seconds left on the shot clock (all training shots with the shot clock on; 1-second groups with at least 100 shots).

Shots taken with 4 seconds or less on the shot clock go in 36.2% of the time, against 49.6% for

shots with 15 seconds or more left. Late in the clock, players are often forced into contested shots off the dribble, while early in the clock there are more transition chances and open looks.

Both graphs visualize important relationships in basketball. Firstly, shots close to the rim such as dunks, tip ins and others have a higher field goal percentage due to their simplicity and how it's harder for defenders to block them. Secondly, make rates are linked to how much time is left in the shot clock. As time goes down, teams and players might take more desperate shots that are harder to make. When there are 22 seconds left on the shot clock rates go up to 70%, this is due to it probably being a fast break situation where players are left wide open. Thus, teams should aim for these situations.

Where shots go in

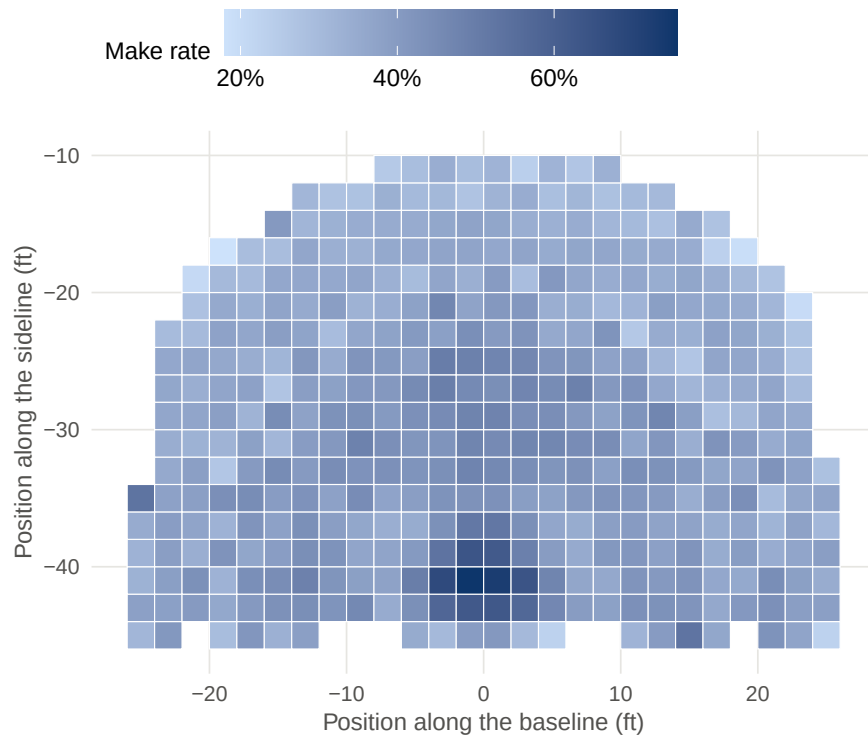


Figure 4: Make rate by court location (2 ft by 2 ft areas with at least 30 shots).

Make rate by closest defender distance, by zone

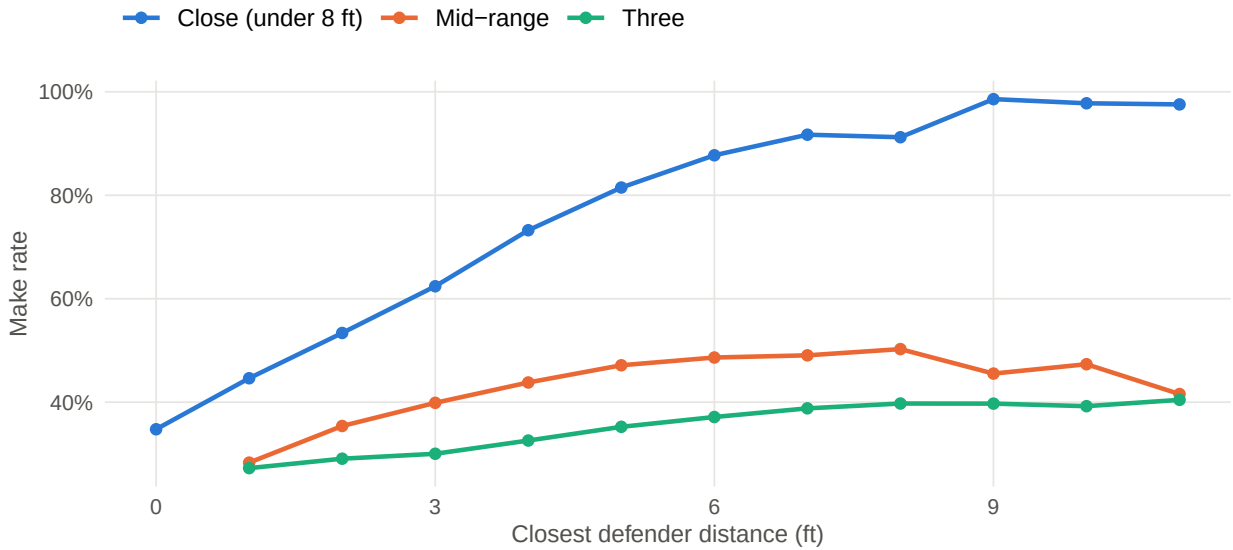


Figure 5: Make rate by distance of the closest defender, for each shot zone.

In every zone, the make rate rises as the closest defender gets further away: the more space the shooter has, the more often the shot goes in. Going from a defender within 2 feet to one 6 feet or more away changes the make rate by 47.5 percentage points close to the rim, 20.3 percentage points in the mid-range and 11.5 percentage points on threes. The value of space therefore depends on the type of shot. That kind of interaction, where one feature changes the effect of another, is what tree models and neural networks can learn and a plain logistic regression cannot. It is also why the chart is split by zone: defenders stand closest on shots near the rim, which go in most often anyway, so mixing all shots together would blur the effect of space.

Space and distance is the strongest factor in correctly predicting the outcome of a shot. The more space there is for a player the less likely he is to commit a mistake in his attempt at the rim. The impact of space is stronger for close shots, this is normal since they are innately easier shots.

3 Methods

3.1 Process:

Below I represent the process I took for my models and how I trained them:

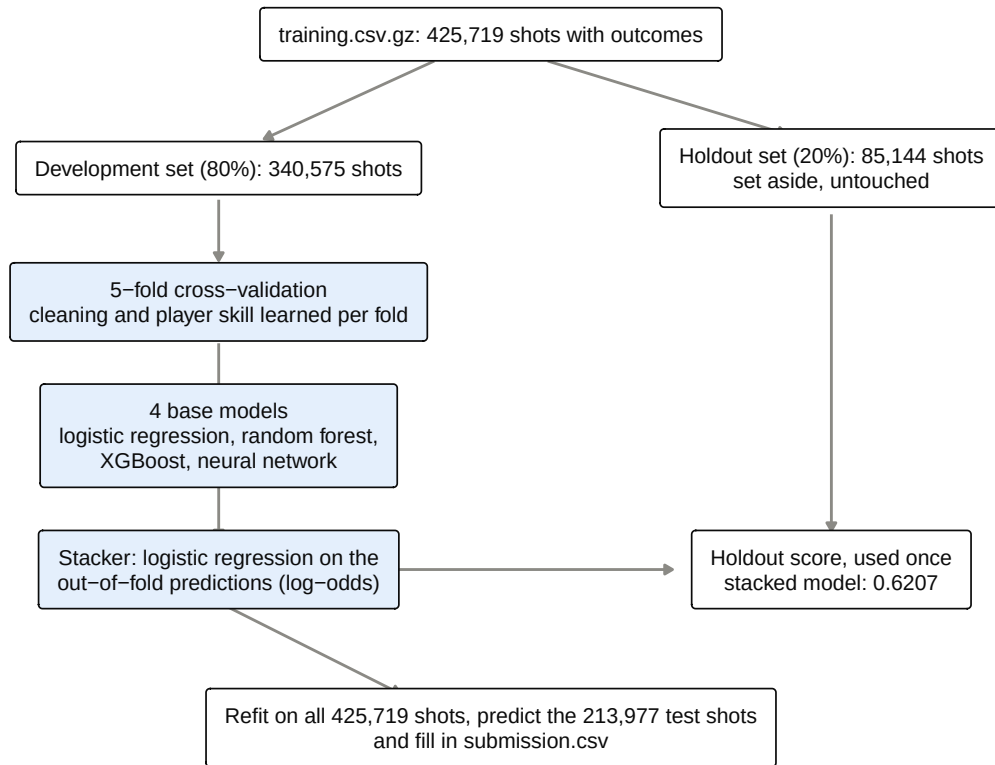


Figure 6: The modelling pipeline. Everything learned from data (cleaning rules, player skill, model weights) is learned inside the folds; the holdout is scored once.

- **80% development, 20% holdout.** The split was stratified by outcome and made before anything else. The holdout was used exactly once, to score the final model.
- **Five folds inside development.** Each model was trained five times, each time on four folds, and predicted the fifth. This gives one out-of-fold prediction for every development shot. All model comparisons and all choices in this paper use these out-of-fold scores.
- **Everything is learned inside the folds.** Cleaning rules (medians, category levels, scaling) and player skill columns were learned from each fold’s training shots only and then applied to that fold’s validation shots. For the holdout they were relearned on all development shots, and for the test file on all training shots.

3.2 Correlated inputs

The cleaning steps are described in Section 2.2. One more check before modelling was how strongly the numeric inputs move together.

Correlation between the numeric inputs

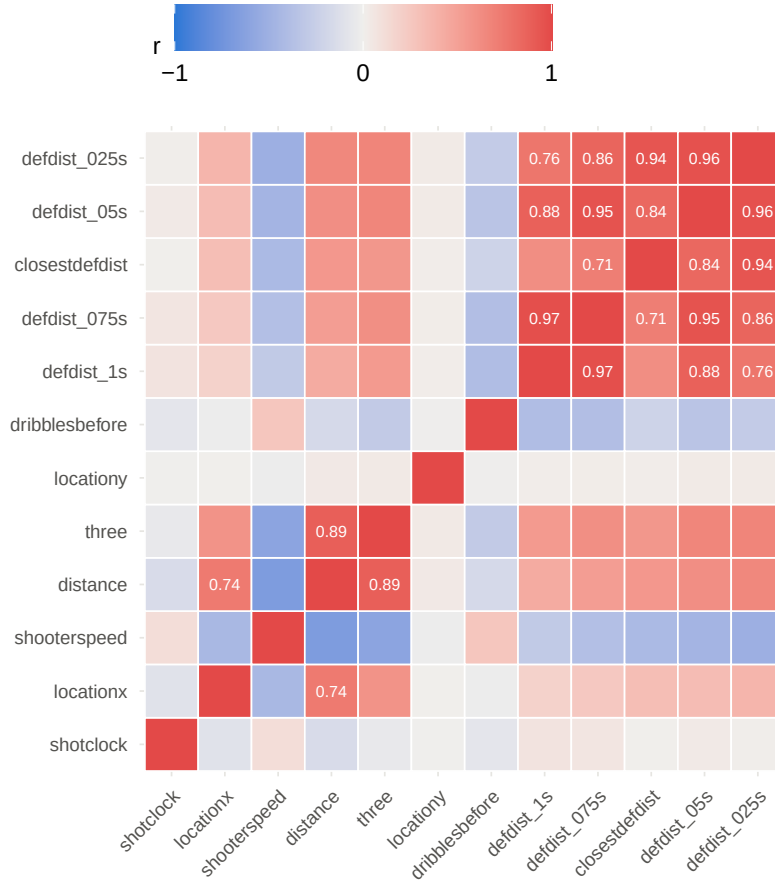


Figure 7: Correlation between the numeric inputs (before cleaning).

The four approach distances are strongly correlated with each other and with the closest defender's distance at release, since a defender who is close one second before the shot is usually still close at release. I kept them and removed only exact duplicates. The tree models, which carry most of the final predictions, are not hurt by correlated inputs, and together the approach columns describe whether the defender was closing out or backing off.

3.3 Player skill: makes above expected

The data identify the shooter and the closest defender, but a raw make rate mixes skill with shot selection: a center's make rate is high because he takes dunks, not because he is a great shooter. I therefore measure skill relative to shot difficulty. Inside each fold, a logistic regression on the shot features alone gives every training shot an expected make probability $\hat{p}_i$. The residual $r_i = y_i - \hat{p}_i$ is positive when a shot beats its expectation (+0.6 for making a 40% shot). For a player j with shots S_j , the skill score is

$$A_j = \frac{\sum_{i \in S_j} r_i}{n_j + m}, \quad m = 100,$$

which behaves like adding 100 average shots to every player’s record. Players with few shots are pulled toward zero (average), in the spirit of empirical Bayes shrinkage (Efron & Morris, 1975). Three scores are computed: per shooter, per shooter in each zone (close, mid-range, three), and per closest defender. For training shots the scores are themselves computed out of fold, so no shot ever sees its own result. A player who never appears in training receives 0.

Raw make rate against makes above expected

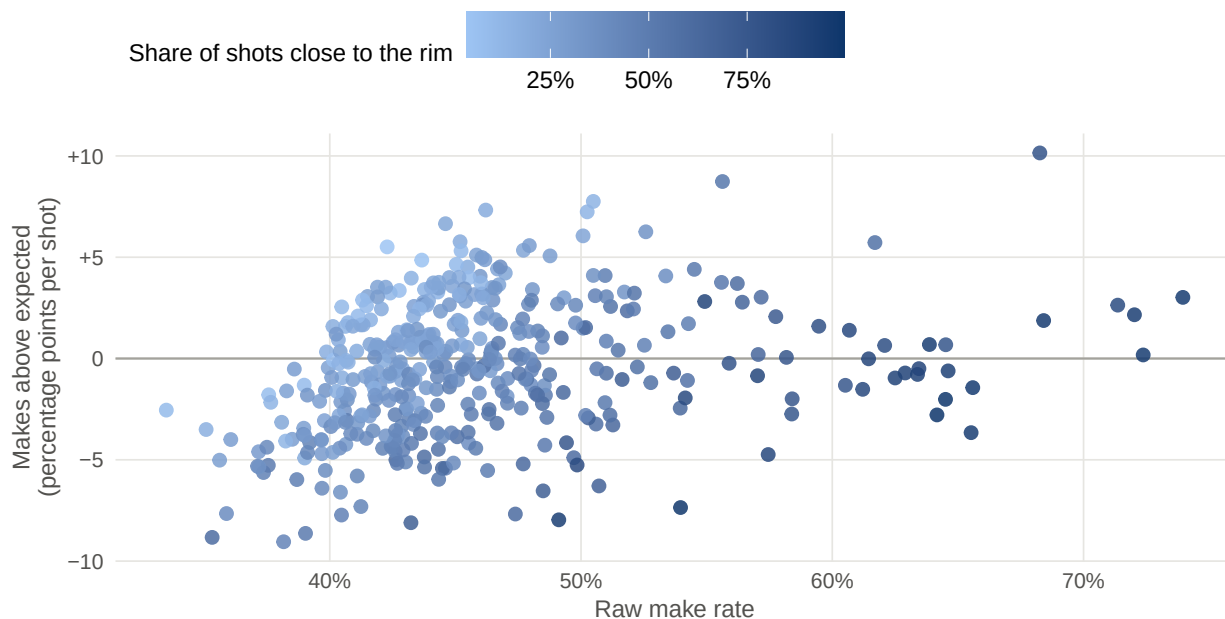


Figure 8: Shooters with at least 200 development shots: raw make rate against makes above expected.

If raw make rates measured skill, the points would form a single rising band. They do not: at the same makes above expected, players who take more of their shots near the rim (darker points) have higher raw make rates, by about 3 percentage points for every extra 10% of their shots taken close to the rim. Their high percentages come partly from easier shots, not only from skill.

The extra variable created follows basketball intuition. Rudy Gobert made 68.2% of his attempts in the 2025/2026 season, whereas Stephen Curry only made 46.8%. If field goal percentage alone described skill, then Rudy Gobert would be a better shooter than Stephen Curry. No basketball analyst would support this decision. The difference in their FG% is due to their shot diet and difficulty. The metric makes above expected removes this difference by comparing every shot with how often an average player would make it under the same circumstances. Rewarding players for hard shots.

3.4 Base models

Four learners with different strengths were trained on the same inputs.

Table 4: Base models and their settings.

Model	Package	Settings
Logistic regression	glm (stats)	all inputs, logit link
Random forest	ranger	300 trees, minimum node size 50, probability forest
XGBoost	xgboost	learning rate 0.02, depth 6, minimum child weight 100, 80% row and column sampling, number of trees chosen by early stopping on a 10% slice
Neural network	nnet	one hidden layer of 10 units, weight decay 10, cross-entropy loss, standardized inputs

- **Logistic regression** is linear on the log-odds scale. It is transparent and serves as the benchmark the other models must beat.
- **The random forest** (Breiman, 2001; Wright & Ziegler, 2017) averages hundreds of trees grown on random samples of shots. It finds curved effects and interactions without being told where to look. The large minimum node size keeps its probabilities smooth.
- **XGBoost** (Chen & Guestrin, 2016) grows trees one after another, each correcting the previous ones’ errors. The slow learning rate and minimum leaf weight stop it from building leaves on a handful of shots.
- **The neural network** (Venables & Ripley, 2002) learns smooth curved relationships that trees can only approximate with steps, which makes its errors different from the tree models’ errors.

3.5 Stacked ensemble

The four models feed a stacked model (Wolpert, 1992): a logistic regression on the log-odds of their out-of-fold predictions,

$$\text{logit} \left(p_i^{\text{stack}} \right) = \beta_0 + \sum_{k \in \{\text{glm}, \text{rf}, \text{xgb}, \text{nn}\}} \beta_k \text{logit} \left(\tilde{p}_{ik} \right),$$

where $\tilde{p}_{ik}$ is model k ’s prediction for shot i , clipped to $[0.001, 0.999]$. Because every $\tilde{p}_{ik}$ comes from a model that never saw shot i , the weights β_k reflect how much each model helps on new shots, not how well it memorized the training shots. The stack was itself scored out of fold, learning its weights on four folds and predicting the fifth. For the holdout and the test file, it was refit on all out-of-fold predictions.

3.6 Evaluation metric and benchmarks

Predictions are scored by log loss,

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)],$$

where y_i is 1 for a make and p_i the predicted probability. Log loss has no natural “good” value, so every model is compared with two benchmarks computed on the same shots: a model that predicts the average make rate for every shot, and one that knows only the shot type.

4 Results

4.1 Cross-validation

Table 5: Out-of-fold log loss on the development shots (lower is better).

Model	Log loss	Improvement over no model
No model (average make rate)	0.6896	0.0%
Shot type only	0.6540	5.2%
Logistic regression	0.6365	7.7%
Random forest	0.6270	9.1%
XGBoost	0.6234	9.6%
Neural network	0.6282	8.9%
Stacked model	0.6233	9.6%

Every model beats both benchmarks. The stacked model scores 0.6233 out of fold, against 0.6896 for knowing nothing and 0.6540 for knowing only the shot type.

Out-of-fold log loss by model

For reference, predicting the average make rate scores 0.6896

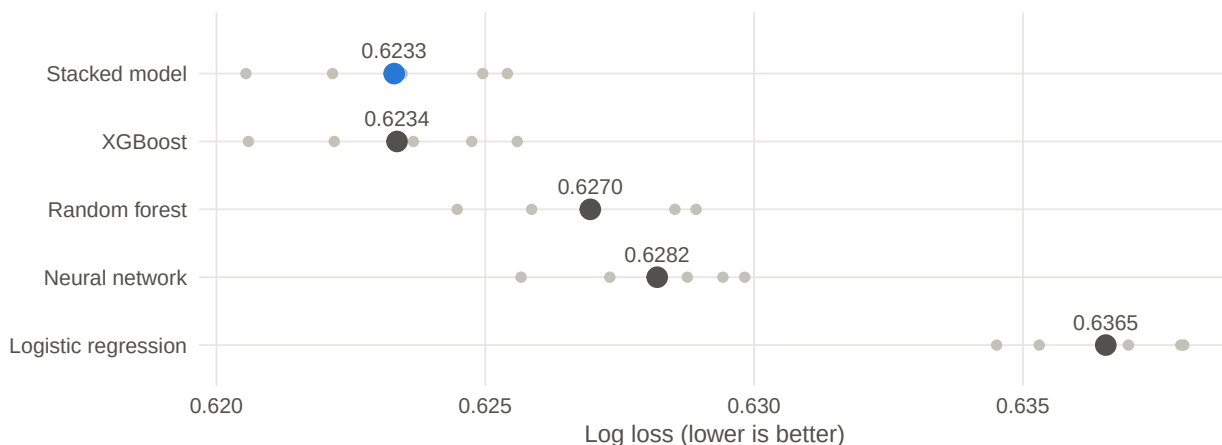


Figure 9: Out-of-fold log loss of each model: large dots are the overall scores, small dots the five folds.

The five folds give similar scores for every model, so the results are not the product of one lucky split. These numbers must be read in context: log loss cannot reach 0 for basketball shots. Even a perfect model would say 45% for a shot that is truly 45%, and that shot will still miss 55%

of the time, impacting the log loss of the model. Most of the remaining log loss is this built-in randomness, so what separates good models is a few thousandths of log loss, not tenths.

4.2 What helped and what did not

I kept a change only if it lowered the out-of-fold log loss. The changes below were tested with XGBoost, the strongest single model.

Table 6: Changes tested with XGBoost (out-of-fold log loss).

Change	XGBoost log loss	Kept
Starting point (shot features only)	0.6261	
Players' raw make rates	0.6256	Replaced
Players' makes above expected	0.6234	Yes
Extra shot features	0.6235	No
Isotonic calibration	Worse	No

Raw make rates barely helped (0.6261 to 0.6256), while makes above expected improved XGBoost to 0.6234, four times the gain. The extra shot features (the closing speed of the closest defender, defender space relative to shot distance, a corner three flag and a shot clock off flag) were reasonable basketball ideas but did not lower the log loss, most likely because the models could already build them from the raw columns. Isotonic calibration made the score worse because the probabilities were already well calibrated (Section 4.4), so the correction only added noise. One pitfall early on was a faulty log loss function that scored the no-model benchmark at 0.655 instead of 0.690; checking every score against that benchmark caught it.

4.3 Holdout evaluation

After every choice had been made with the folds, all four base models were refit on the full development set, the stacker was refit on all out-of-fold predictions, and the result was scored once on the 85,144-shot holdout.

Table 7: Log loss on the holdout (scored once, lower is better).

Model	Log loss	Improvement over no model	Standard error
No model (average make rate)	0.6896	0.0%	
Stacked model	0.6207	10.0%	0.0011
Logistic regression	0.6348	8.0%	0.0010
Random forest	0.6252	9.3%	0.0011
XGBoost	0.6207	10.0%	0.0011
Neural network	0.6264	9.2%	0.0011

The stacked model scores **0.6207** on the holdout, a 10.0% improvement over the no-model benchmark (0.6896). It ties XGBoost (0.6207); the gap of 0.00003 is far smaller than the noise in a sample of this size. Per shot, the stack's log loss differs from XGBoost's by 0.00003 on average,

with a standard error of 0.00007; the two are statistically indistinguishable on the holdout. The stack’s small out-of-fold advantage therefore did not carry over to the holdout, but it did not reverse either. I kept the stacked model because it was chosen from the fold scores before the holdout was opened; switching models after seeing the holdout would turn the holdout into another tuning set.

Every model’s holdout score is within 0.0026 of its out-of-fold score (0.6233 out of fold for the stack), which confirms that the fold scores were honest and that nothing leaked from validation shots into training. The holdout scores are, if anything, slightly lower. This is expected: the final models learned from all development shots, a quarter more data than each fold’s training set.

Table 8: Weights of the stacked model (coefficients on each model’s log-odds).

Input	Weight
Intercept	0.005
Logistic regression	-0.029
Random forest	0.179
XGBoost	0.755
Neural network	0.127

The stacker puts a weight of 0.75 on XGBoost, far more than on the random forest (0.18) and the neural network (0.13). Logistic regression receives a weight close to zero (-0.03): once the other three are known, it adds no information. The forest and the network each add a small correction, which is why the stack and XGBoost end up so close.

4.4 Calibration

For log loss, calibration (predicted probabilities matching observed frequencies) matters as much as ranking shots correctly (Niculescu-Mizil & Caruana, 2005). The holdout shots were split into 20 equal-sized groups by the stacked model’s predicted probability.

Table 9: Calibration on the holdout: 20 equal-sized groups of shots by predicted probability.

Predicted probability bin	Shots	Average predicted	Observed make rate
[0.0271,0.267]	4,258	0.207	0.212
(0.267,0.304]	4,257	0.288	0.299
(0.304,0.324]	4,257	0.315	0.323
(0.324,0.339]	4,257	0.332	0.330
(0.339,0.351]	4,257	0.345	0.341
(0.351,0.362]	4,257	0.356	0.365
(0.362,0.372]	4,258	0.367	0.359
(0.372,0.383]	4,257	0.377	0.374
(0.383,0.395]	4,257	0.389	0.389
(0.395,0.407]	4,257	0.401	0.389
(0.407,0.421]	4,257	0.414	0.414

Predicted probability bin	Shots	Average predicted	Observed make rate
(0.421,0.437]	4,257	0.429	0.417
(0.437,0.457]	4,257	0.447	0.444
(0.457,0.483]	4,258	0.469	0.474
(0.483,0.516]	4,257	0.499	0.493
(0.516,0.56]	4,257	0.537	0.527
(0.56,0.623]	4,257	0.589	0.570
(0.623,0.724]	4,257	0.670	0.677
(0.724,0.883]	4,257	0.801	0.813
(0.883,0.994]	4,258	0.945	0.951

Predicted against observed make rate (holdout)

Dashed line = perfect calibration

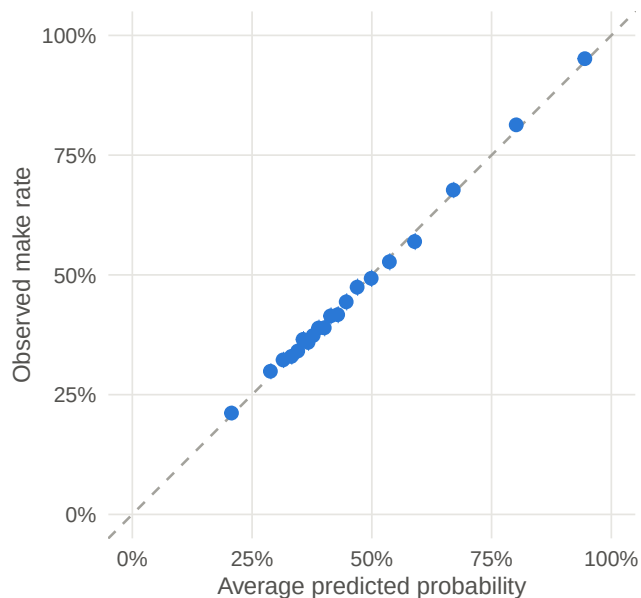


Figure 10: Calibration of the stacked model on the holdout. Each point is one group from the table above; bars show two standard errors.

Across the 20 groups, the largest gap between the average predicted probability and the observed make rate is 0.020, or 2 percentage points. The points follow the diagonal across the whole range, so the model's probabilities can be taken at face value, which is also why isotonic calibration did not help.

4.5 Distribution of predicted probabilities

Distribution of predicted shot probabilities (holdout)

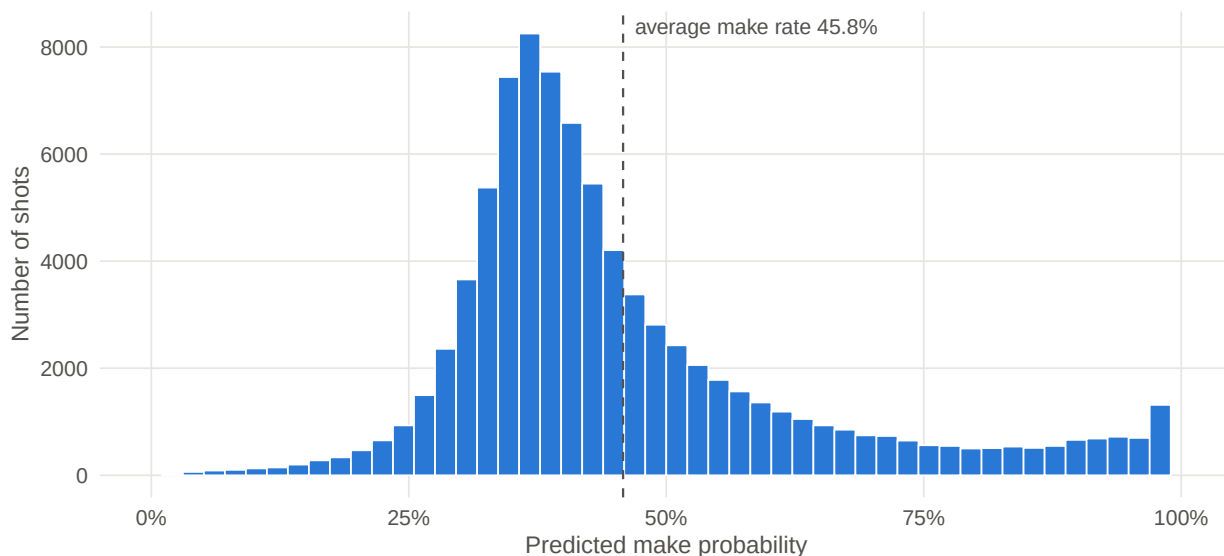


Figure 11: Distribution of the stacked model’s predicted make probabilities on the holdout shots.

The middle half of the holdout shots receive a probability between 35.1% and 51.6%. Only 16.6% of shots are predicted above 60% and 0.4% below 10%. The model rarely makes extreme predictions, which is appropriate for an outcome as noisy as a basketball shot.

Table 10: Predicted make probabilities for the 213,977 shots in the test file (final model refit on all training shots).

Statistic	Predicted probability
Minimum	0.024
First quartile	0.349
Median	0.405
Mean	0.457
Third quartile	0.514
Maximum	0.995

On the test file, the average predicted probability is 0.457, practically identical to the make rate in the development data (0.458), so the test shots were cleaned and scored consistently. The median (0.405) sits below the mean because the distribution has a long upper tail of high-probability shots near the rim.

4.6 What the model relies on

A stacked model has no single importance ranking of its own. Since XGBoost carries most of the stack’s weight, its importance ranking is the best available view of what drives the final predictions.

XGBoost feature importance (top 15)

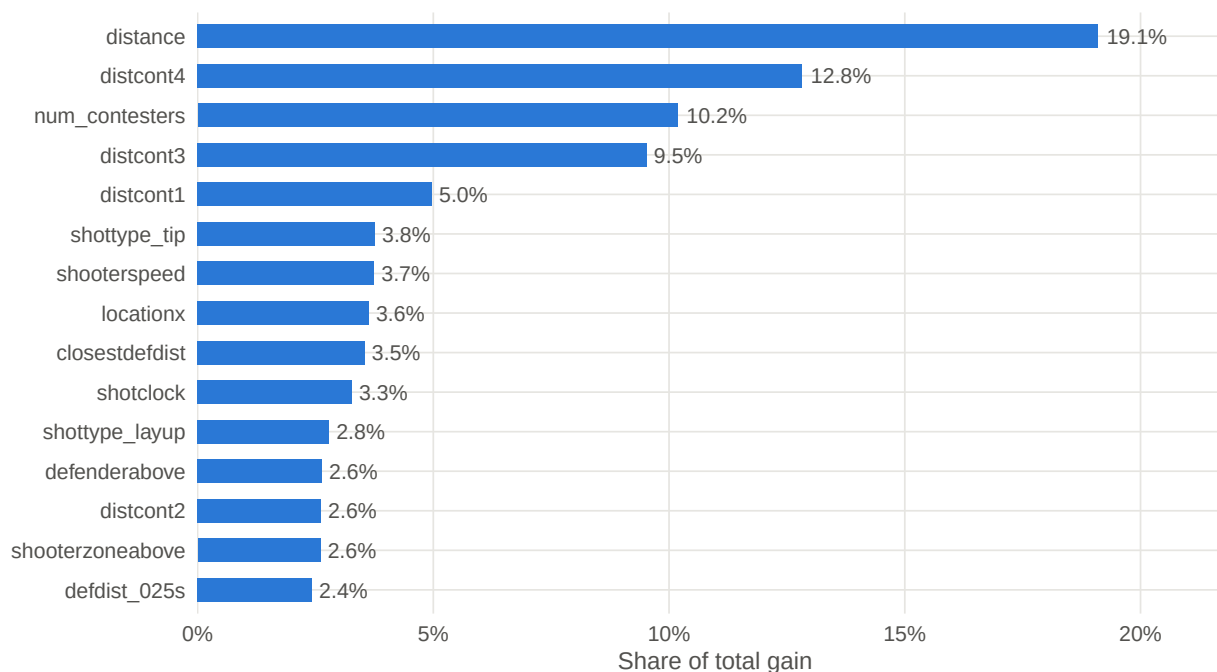


Figure 12: The 15 most important inputs to XGBoost, by share of total gain.

Gain measures how much each input improved XGBoost's predictions across all its trees. The three most important inputs are distance, distcont4, num_contesters, which together account for 42.1% of the total gain.

As we mentioned before, distance to the rim is the most important predictor on the outcome of a shot. A layup and a full court shot don't represent the same value because of how much harder one is to the other. Furthermore, distcont4 and distcont3's values were mainly replaced with the distance of the shot, thus following the early trend. Number of defenders represents an important factor because of the possibility of one player blocking or altering the shot. Therefore, the increase in defenders has a direct impact on the probability of a shot going in.

5 Discussion: from probabilities to decisions

5.1 Expected points and shot selection

The most direct use of a make probability is **expected points**: the probability times the shot's point value. A 36% three is worth 1.08 points per attempt, while a 42% mid-range jumper is worth only 0.84.

Table 11: Actual and predicted make rates, and expected points per shot, by zone (development shots, out-of-fold predictions).

Zone	Share of shots	Make rate	Predicted	Expected points per shot
Close (under 8 ft)	37.9%	58.4%	58.6%	1.17
Mid-range	20.6%	42.1%	42.0%	0.84
Three	41.5%	36.2%	36.0%	1.08

Expected points per shot by court location

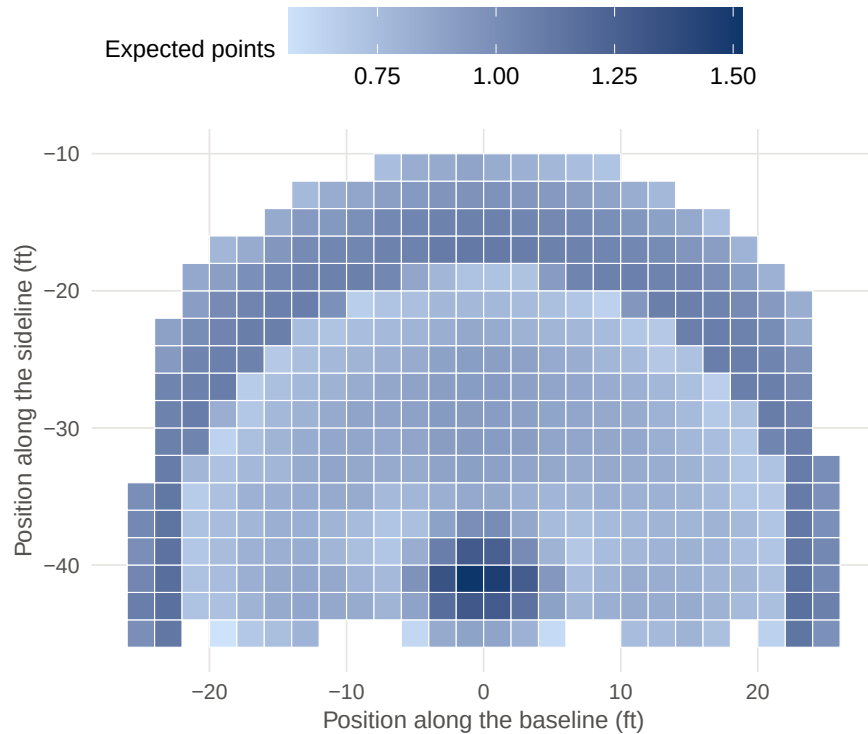


Figure 13: Average expected points per shot by court location (2 ft by 2 ft areas with at least 30 shots).

On offense, this turns shot selection into a comparison of expected points. Points per possession come from how often a team shoots and how many points each shot is expected to produce, so a team can:

- **Grade shot quality separately from shot making.** Comparing a team's actual points with its expected points shows whether it is getting good looks and missing them (likely to improve) or making difficult shots (likely to regress).
- **Design plays around the most valuable spots.** The expected points map shows which areas of the floor produce the most points per attempt.
- **Evaluate players fairly.** Makes above expected separates a shooter's skill from the difficulty of his shots, so a player who takes hard shots is not penalized for a lower raw percentage.

The math would dictate taking rim attempts and three pointers over any other type of shot. However, there are players who are exceptions to this rule. SGA is a clear example, his mid range game is so effective and lethal that it opens up the floor for his teammates and usually tends to higher quality shots.

Information such as this is highly useful for basketball teams, if a scouting team can prepare the coaching staff with an accurate estimation of the expected point per possession and specific shots for opponents then gameplans can be made.

5.2 Defense

The same numbers work in reverse on defense. A defense wants to push opponents into low expected points shots and away from high ones.

Expected points per shot by closest defender distance

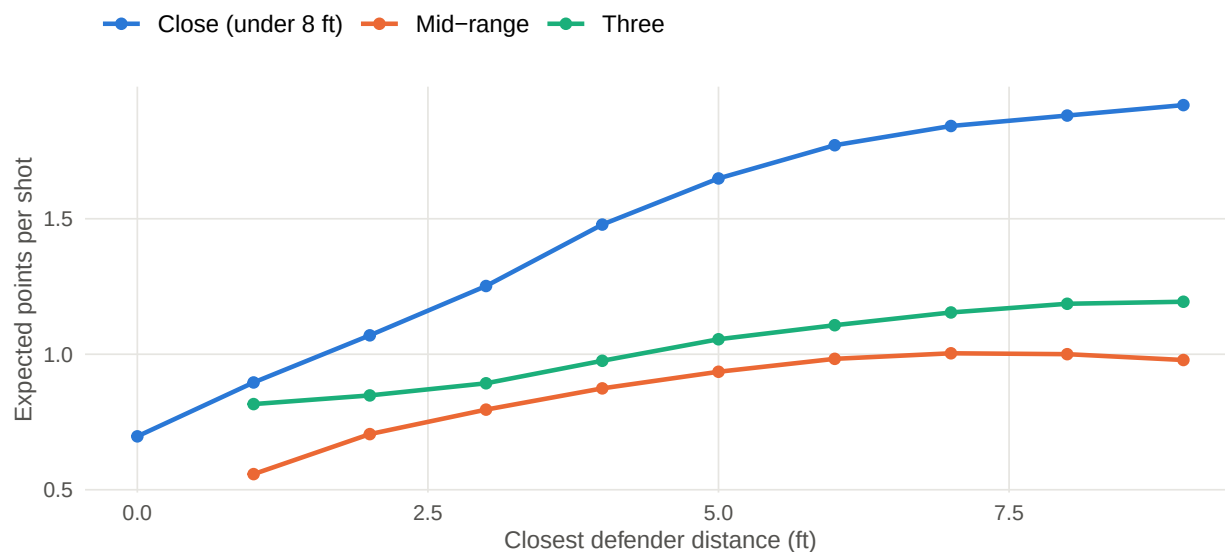


Figure 14: Average expected points per shot by distance of the closest defender, for each shot zone.

Averaged over the development shots, a three with the closest defender within 2 feet is worth 0.82 expected points, against 1.16 when the defender is 6 feet or more away. These are averages of the model's predictions rather than controlled experiments, since open and tight shots also differ in other ways, but they show how a defense can use the model:

- **Decide which shots to concede.** If a defense has to give up something, the zone table and the map show which shots cost the fewest points.
- **Value closeouts.** The gap between tight and open threes puts a point value on closing out, which helps decide when to help off a shooter and when to stay home.
- **Evaluate defenders.** The defender skill column (makes above expected allowed) shows which defenders lower their opponents' make rates beyond what the shot type and distance predict.

- **Scout opponents.** Combining an opponent’s shot chart with each shooter’s makes above expected by zone shows which players to run off the line and which to let shoot.

This graph would support the already adopted defensive scheme of sagging off a worse shooter to help around the half court. If the team can send an extra defender to an offensive player, then the team limits the times where an offensive player finds himself open. Furthermore, this supports peel switching to not leave any player open.

6 Limitations

- **The “Curry factor”.** Player skill is shrunk toward the average and estimated from shots similar to those of an average player. Truly exceptional shooters, such as one who makes pull-up threes from 30 feet at a high rate, are therefore underrated, especially on the rare shots that only they attempt.
- **Skill is treated as fixed.** The model cannot see streaks, injuries, or improvement over a season.
- **The data do not describe the shot itself.** There is no information on release angle, arc, ball speed, or the shooter’s balance and body orientation.
- **Defense is measured only by distance.** The data give how far defenders are, but not their height, wingspan, hand position, or the angle they approach from.
- **Missing contest distances.** Filling them with the shot distance turned the third and fourth contest distances into copies of shot distance on most shots (Section 2.2). The number of contesters tells the models which values are real, but a fill meaning “no contest” would have been cleaner.
- **Context is limited.** The score margin, the time left in the game, fatigue, and how the shot was created (off a pass or off the dribble) are not in the data.

7 Future work

- Ball tracking (release angle, arc, release height) and shooter body orientation.
- Defender attributes and a direction for the closest defender, not just a distance.
- Assist and pass information, to separate catch-and-shoot shots from off-the-dribble ones.
- Skill estimates that change over time, weighting recent shots more heavily.
- More tuning of XGBoost and more varied base models, so the stack has more disagreement to work with.

8 Conclusion

It is impossible to reach a log loss of 0 when predicting basketball shots; the randomness of the sport puts a floor under every model. Within that limit, the stacked model scores 0.6233 out of fold and 0.6207 on the untouched holdout, a 10.0% improvement over predicting the average make rate, with well calibrated probabilities. The largest gains came from a clean, leak-proof pipeline, strong tree models, and measuring player skill as makes above expected. Stacking four models added little over XGBoost alone because the models learned mostly the same patterns, and the stack’s

weights make that visible. More importantly, the model turns every shot into an expected value, and thinking in expected values rather than makes and misses is what produces better decisions over a season.

This work represents a sample of what I could do for the Oklahoma City Thunder, it would be amazing!

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).
- Efron, B., & Morris, C. (1975). Data analysis using Stein’s estimator and its generalizations. *Journal of the American Statistical Association*, 70(350), 311–319.
- Kuhn, M., & Johnson, K. (2019). *Feature Engineering and Selection: A Practical Approach for Predictive Models*. CRC Press.
- Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632).
- Venables, W. N., & Ripley, B. D. (2002). *Modern Applied Statistics with S* (4th ed.). Springer.
- Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241–259.
- Wright, M. N., & Ziegler, A. (2017). ranger: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1), 1–17.

Appendix: reproducibility

All code is in `project_code.Rmd`. Knitting it reads `training.csv.gz` and `testing.csv.gz`, runs the cross-validation, the holdout check and the final refit, fills in `submission.csv`, and saves `writeupdata.rds`, from which this paper is built. Random seeds are fixed throughout (the 80/20 split uses seed 12345 and the folds seed 678), so the results can be reproduced exactly. The analysis uses R with the packages `dplyr`, `ggplot2`, `rsample`, `recipes`, `ranger`, `xgboost` and `nnet`.